

Fantasy Premier League Report

Team 1

October 24, 2025

1 Introduction

Fantasy Premier League (FPL) is an interactive online football management game in which participants create and manage virtual squads made up of real Premier League players. Every player earns points based on their actual match performance, such as scoring goals, keeping clean sheets, or providing assists. With millions of active users worldwide, FPL has evolved into a highly competitive strategic game where success relies on informed decision-making rather than luck alone.

As football analytics continues to advance, data plays a central role in how FPL managers evaluate player performance, plan weekly transfers, and choose starting lineups. Statistical indicators like form, expected goals, and fixture difficulty help managers predict future points more accurately. This combination of real-life sports performance and strategic data analysis transforms FPL into a valuable domain for applying machine learning and predictive modelling.

In this project, we focus on how structured data from historical Premier League seasons can be leveraged to understand performance trends and predict upcoming player points. The insights gained from such analysis provide fantasy managers with a competitive advantage in building stronger teams and optimizing their decisions throughout the season.

2 Dataset Overview

Before performing analysis or developing a predictive model, it is essential to understand the structure and quality of the dataset.

2.1 Dataset Summary

- **Total Rows:** 96,169
- **Total Columns:** 37

2.2 Column Information

Feature Categories:

- **Identifiers:** season_x, name, element, GW
- **Player Role:** position, team_x
- **Match Context:** was_home, opponent_team, opp_team_name, fixture, team_a_score, team_h_score, kickoff_time, round
- **Performance Metrics:** minutes, goals_scored, assists, clean_sheets, saves, penalties_saved, penalties_missed, own_goals, yellow_cards, red_cards
- **Fantasy Scoring:** total_points, bonus, bps, threat, creativity, influence, value
- **Manager Behavior:** selected, transfers_in, transfers_out, transfers_balance

Data Types:

- `int64`: 24 columns
- `float64`: 6 columns
- `object`: 6 columns
- `bool`: 1 column

2.3 Missing Values

Table 1: Columns with Missing Values

Column	Missing Count	Missing Percentage
<code>team_x</code>	19,852	20.64%

Only one column shows missing values of notable size, indicating generally good data quality.

3 Data Cleaning

3.1 Removing Columns

Certain features were removed from the dataset because they were either redundant, caused data leakage, or did not contribute meaningful predictive value to our machine learning model. The removed columns and reasons are summarized below:

- **ict_index**: A composite metric combining Influence, Creativity, and Threat into a single score.
Reason for removal: The individual components (Influence, Creativity, Threat) are already included as separate features. Keeping this combined metric introduces redundancy.
- **kickoff.time**: The date and time when the match started.
Reason for removal: The exact timestamp does not offer meaningful predictive power for future player performance and adds unnecessary complexity.
- **round**: Represents the gameweek number.
Reason for removal: The dataset already contains the column **GW**, which provides the same information.
- **selected**, **transfers_balance**, **transfers_out**: These columns reflect manager behavior and player popularity.
Reason for removal: They fall outside the scope of this project since we focus solely on performance-based features rather than popularity trends.

Impact on Model: Removing these features prevents potential data leakage and avoids unnecessary redundancy. Prior to removal, there was a risk that the model could:

- Overfit the training data by learning patterns that do not generalize to future predictions
- Rely on features that would not be realistically available or useful when forecasting upcoming gameweek points

3.2 Handling Missing Team Information

Some rows in the dataset contain missing values in the `team_x` column. Since a player's team has a strong influence on their performance and available statistics, missing team information reduces the usability of team-related features. However, removing all rows with missing teams would lead to a significant loss of valuable data.

Solution Strategy: We recover missing team values using fixture information. Each fixture consists of exactly two teams playing against each other. By utilizing the opponent team and fixture identifier, we can infer the missing team value.

- Group rows by (**season, gameweek, fixture**) to identify both teams participating in a match.
- For each player:
 - If the opponent team is A, then the player’s team must be B.
 - If the opponent team is B, then the player’s team must be A.

This approach enables the recovery of **team_x** values without discarding any data.

Validation: We verify correctness by manually checking several updated samples to ensure that the inferred team corresponds to the player’s actual club during that fixture. Example rows are inspected to confirm the correct mapping between opponent and team.

3.3 Checking for Duplicate Rows

Duplicates in the dataset can negatively impact the performance and reliability of the model. It is therefore essential to detect and handle them properly.

Why Check for Duplicates?

- **Inflated performance:** Repeated entries artificially boost model accuracy during evaluation.
- **Data leakage:** The same instance appearing in both training and testing splits misleads the model.
- **Overfitting:** The model memorizes duplicate values instead of learning meaningful patterns.

Types of Duplicates Identified:

- **Exact duplicates:** All columns are identical.
- **Partial duplicates:** Same player in the same season and gameweek appears more than once.

Each player should only have a single entry per gameweek. Multiple rows for the same player and gameweek indicate data quality issues that must be corrected prior to model training in order to ensure clean and unbiased learning data.

3.4 Position Standardization

The **position** column exhibited inconsistencies where goalkeepers were represented by both **GK** and **GKP**. Such discrepancies can negatively affect the model’s ability to learn accurate patterns based on player positions. Standardizing categories improves feature quality and analytical clarity.

Before Standardization:

```
position
MID    36768
DEF    31593
FWD    11914
GK      9900
GKP      80
Name: count, dtype: int64
```

Issue: Inconsistent representation of goalkeepers (GK and GKP).

Standardization Mapping:

- GK, GKP → GK (Goalkeeper)
- DEF → DEF (Defender)
- MID → MID (Midfielder)
- FWD → FWD (Forward)

This ensures consistent position categories across the entire dataset.

After Standardization:

```

position
MID      36768
DEF      31593
FWD      11914
GK        9980
Name: count, dtype: int64

```

Unique standardized positions:

```
['DEF', 'MID', 'GK', 'FWD']
```

This update preserves all rows and improves feature consistency for both analysis and predictive modeling.

3.5 Column Data Type Validation

Ensuring that each feature has a consistent data type is essential for correct model training and reliable statistical analysis. A full column-wise inspection was performed to detect type mismatches.

Results of Data Type Check:

- **String or null:** `season_x`, `name`, `position`, `team_x`, `opp_team_name`
- **Numeric:** `assists`, `bonus`, `bps`, `clean_sheets`, `creativity`, `element`, `fixture`, `goals_conceded`, `goals_scored`, `influence`, `minutes`, `opponent_team`, `own_goals`, `penalties_missed`, `penalties_saved`, `red_cards`, `saves`, `team_a_score`, `team_h_score`, `threat`, `total_points`, `value`, `yellow_cards`, `GW`
- **Boolean:** `was_home`

Summary: No column exhibited incorrect or mixed data types.

Why Data Type Validation Matters:

- Numeric columns must be numeric to support model training and statistical operations
- Boolean columns must remain boolean for correct logical processing
- Mixed or incorrect types lead to unpredictable model behavior and training failures

Handling Incorrect Types: If any incorrect data types had been detected, affected rows would be removed to ensure dataset reliability because:

- Incorrect types cannot always be safely converted
- They often indicate deeper data quality problems
- It is preferable to lose a small number of rows rather than compromise model performance

4 Calculating Form

4.1 Form Feature Engineering

Definition:

$$\text{Form} = \frac{\text{Average of previous 4 gameweeks total_points}}{10}$$

This metric captures how well a player has performed recently, which is a strong indicator of their future performance. Players scoring consistently high points in recent matches tend to maintain strong fantasy results, while those with low recent performance are more likely to struggle.

Why Use the Previous 4 Gameweeks?

- Balances recency and stability
- 1 to 2 matches cause volatility (one bad match skews results)

- More than 8 matches reacts too slowly to changing form
- 4 matches is widely used as a standard measure in fantasy football analytics

Why Divide by 10?

- Normalizes the scale for model training
- Raw points often range from 0 to 20 per gameweek
- Form becomes a more controlled range from 0 to 2
- Helps neural networks learn more efficiently

Implementation Details:

- Form is calculated per player per season
- Only points from previous weeks are used to avoid data leakage
- Calculated even when fewer than 4 past gameweeks exist (beginning of season)

Visualization: The form trajectory is plotted for sample players across each season to check:

- Smooth evolution without random spikes
- Low values in first gameweeks due to limited history

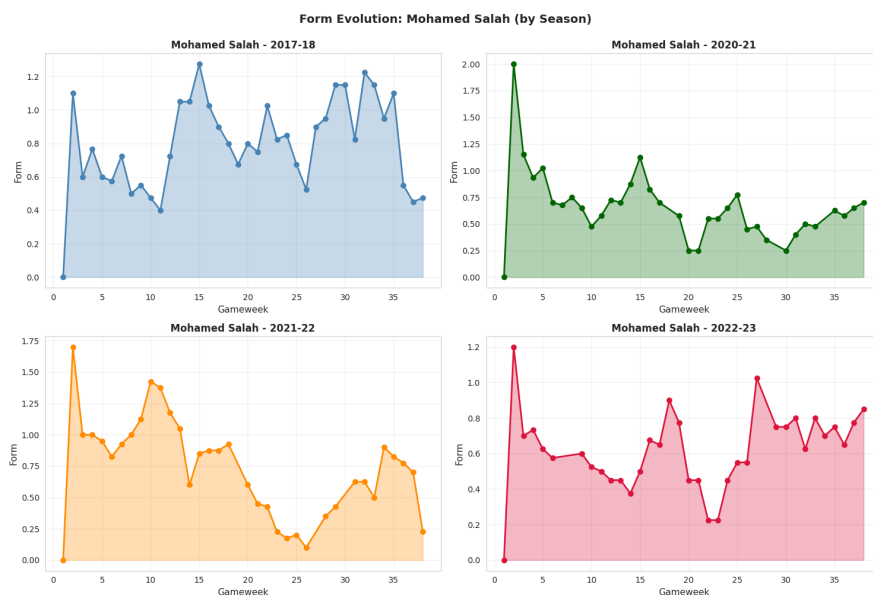


Figure 1: Form evolution example for Mohamed Salah

Example: Mohamed Salah Form Calculation

Player	Season	GW	total_points	form
Mohamed Salah	2017-18	1	11	0.000
Mohamed Salah	2017-18	2	1	1.100
Mohamed Salah	2017-18	3	11	0.600
Mohamed Salah	2017-18	4	1	0.767
Mohamed Salah	2017-18	5	10	0.600
Mohamed Salah	2017-18	6	7	0.575
Mohamed Salah	2017-18	7	2	0.725
Mohamed Salah	2017-18	8	3	0.500
Mohamed Salah	2017-18	9	7	0.550
Mohamed Salah	2017-18	10	4	0.475

Rows with form equal to zero: 38,427 This is expected early in each season due to lack of previous match history.

Why This Feature Matters:

- Captures momentum and confidence
- Reflects short term trends that raw totals cannot show
- Strong predictive power for upcoming points

5 Which Positions Score the Most

Why This Matters: Understanding how different positions contribute to total fantasy points helps guide optimal team strategies. If certain positions consistently score higher, fantasy managers should allocate more budget and prioritise transfers accordingly.

Analysis Approach:

- Group players by position
- Calculate total and average points for each position
- Visualise using a bar chart to highlight differences

Statistical Summary:

Table 2: Position Performance Analysis

Position	Total Points Sum	Average Points	Player Count
FWD	18874	1.58	11914
MID	53822	1.46	36768
DEF	40901	1.29	31593
GK	11649	1.17	9980

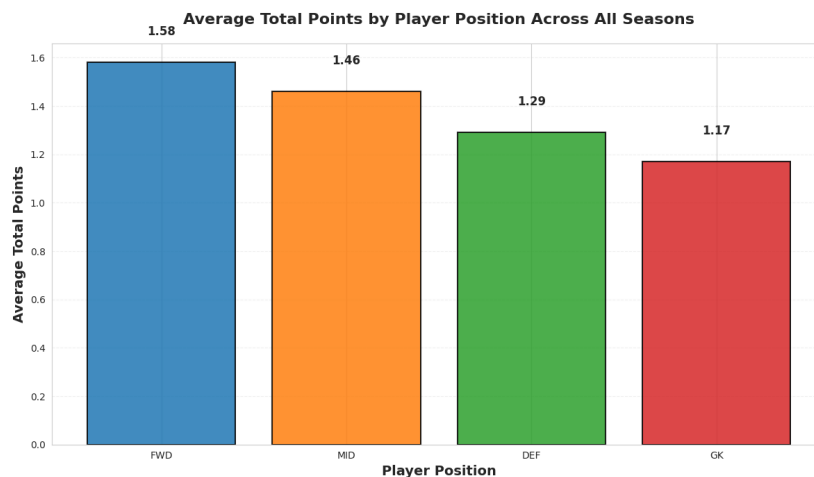


Figure 2: Average Points per Position

Interpretation:

- **Best performing position: Forwards** (1.58 avg points)
- Forwards earn most points due to:
 - Higher scoring frequency (goals and assists)
 - Additional bonus points for attacking contributions

- More consistent involvement in key offensive actions
- Midfielders follow closely due to their dual role in goals and assists
- Defenders and goalkeepers rely heavily on clean sheets which occur less frequently

Overall, the **position** feature plays an important role in predicting player performance. The model will learn typical scoring patterns associated with each position, improving prediction accuracy.

6 Form vs. Total Points: Top 5 Player Analysis

This section investigates whether the top-performing players in terms of total points are also the players who maintain the strongest recent performance (form). If a strong relationship exists, the *form* feature is expected to be highly valuable for upcoming points prediction.

Why This Analysis Matters:

- Stable form indicates predictable future performance
- Total points measure long-term performance over the entire season
- Comparing both metrics validates whether recent form is a strong predictor

Metrics Compared:

- **Form:** Average of the previous 4 gameweeks, normalized by 10
- **Total Points:** Cumulative performance over the full season

Analysis Method:

- Identify the top 5 players by total points in the 2022–23 season
- Identify the top 5 players by average form in the same season
- Compare overlaps between the lists
- Visualize performance using line charts and comparison bar charts

6.0.1 Top 5 Players in the 2022–23 Season

1. By Total Points:

- Harry Kane: 258 points
- Brandon Williams: 247 points
- Axel Tuanzebe: 219 points
- Granit Xhaka: 202 points
- Alfie Mawson: 186 points

2. By Average Form:

- Brandon Williams: 0.76
- Harry Kane: 0.64
- Axel Tuanzebe: 0.62
- Granit Xhaka: 0.55
- DeAndre Yedlin: 0.54

Overlap: 4 out of 5 players appear in both lists (*Kane, Williams, Tuanzebe, Xhaka*)

This suggests that players who are strong performers over the season are also frequently in good form.

6.0.2 Visualization and Insights



Figure 3: Form Evolution of Top 5 Players (2022–23 Season) and Total Points per Gameweek for Top 5 Players (2022–23)

Key Insights:

- Brandon Williams shows early explosive form but becomes volatile later
- Harry Kane demonstrates consistent and stable form throughout the season
- Axel Tuanzebe has a major peak around GW27 (likely a high-scoring match)
- Xhaka and Mawson show declining form in the second half of the season
- Form can be erratic because performance varies weekly in FPL

Key Insights:

- Extreme point volatility from one week to the next
- Most weeks result in 2–8 points for top players
- Occasional massive spikes (20+ points) due to goals, assists, and bonus points
- These rare hauls make total points difficult to predict directly

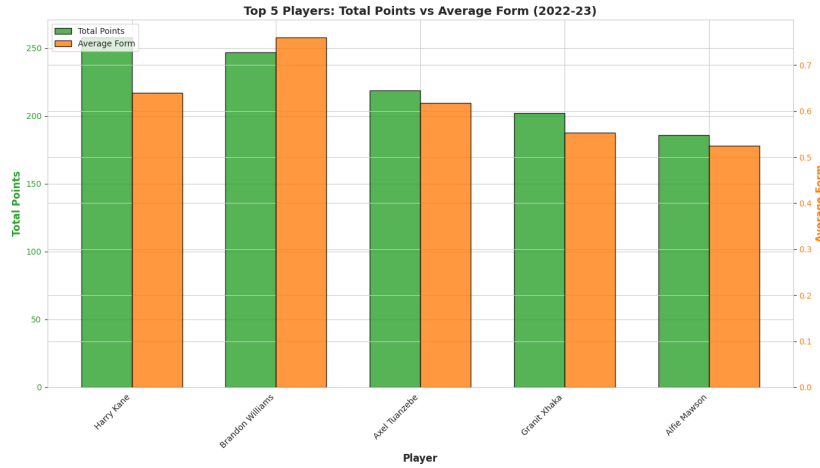


Figure 4: Top 5 Players: Total Points vs. Average Form (2022–23 Season)

Key Insights:

- Brandon Williams leads both total points and average form
- Harry Kane is slightly behind in form but keeps strong season-long performance
- Mawson underperforms in both metrics (limited minutes or lower role)
- Form is a strong but not perfect proxy for overall performance

6.0.3 Conclusion

- Significant overlap between top players by form and total points (4 out of 5)
- Form captures player momentum and short-term performance boosts
- Total points reflect long-term consistency

Final Interpretation: Form is a reliable and important feature that the model should prioritize, but combining form with long-term season performance produces the best insight for predicting upcoming fantasy points.

7 Target Variable Creation

To enable supervised learning for predicting next Gameweek outcomes, two target variables were created: `upcoming_total_points` and `upcoming_blank`. These targets align current Gameweek features with the player’s performance in the following Gameweek.

- **`upcoming_total_points`:** Represents the total FPL points the player will score in the next Gameweek. It is computed using a one-step forward shift within each player and season group.
- **`upcoming_blank`:** A binary classification target that equals 1 if the player scores zero points in the next Gameweek and 0 otherwise.

This approach directly aligns with Fantasy Premier League mechanics, where the primary goal is to maximize future points. It ensures a realistic predictive setup, using only information available before the Gameweek being predicted.

Why this method is effective

- Aligns feature data with real future outcomes, making the problem setup natural for prediction.
- Prevents data leakage by shifting data within each player-season rather than across players or seasons.
- Provides flexibility to run both **regression** (predict points) and **classification** (predict blank vs non-blank).
- Handles Premier League schedule logic by identifying when a player has **no fixture** in the upcoming week.

Handling Season Boundaries

For the last available Gameweek of each player in each season, future information does not exist. These rows produce missing values after the shifting operation and are removed, because training examples without labels cannot be used for supervised learning.

7.1 Dataset Statistics After Target Creation

- Valid rows after shifting: **87,473**
- Rows dropped (final week per player in each season): **2,782**
- Blank rate (upcoming_blank = 1): **54.40%**
- Final dataset size: **87,473 rows**

7.2 Example Output (Darren Randolph)

name	GW	total_points	upcoming_total_points	upcoming_blank
Darren Randolph	1	0	0.0	1
Darren Randolph	2	0	0.0	1
Darren Randolph	3	0	0.0	1
Darren Randolph	4	0	0.0	1
Darren Randolph	5	0	0.0	1
Darren Randolph	6	0	0.0	1
Darren Randolph	7	0	0.0	1
Darren Randolph	8	0	0.0	1
Darren Randolph	9	0	0.0	1
Darren Randolph	10	0	0.0	1

7.3 Importance of upcoming_blank

upcoming_blank signals whether the player has a fixture in the next Gameweek. If a player does not have a match scheduled, they cannot earn points. This feature ensures the model avoids overpredicting points for unavailable players, improving both accuracy and practical usability in FPL decision-making.

8 Enhanced Feature Engineering

Player performance is dynamic and changes as a result of fitness, fixtures, and tactical adjustments. To capture these evolving patterns, eight engineered features were created. These features help the model detect short-term improvements or declines in performance, measure playing consistency, and reduce noise from random high-scoring matches.

- **weighted_form**: A weighted moving average of points over the last four Gameweeks with weights (0.1, 0.2, 0.3, 0.4). This emphasizes recent matches more strongly, allowing the model to identify players who are currently performing well. It distinguishes rising form from performances that occurred further in the past.

- **goals_last_4:** Total number of goals scored in the previous four matches. This quantifies recent attacking output and identifies forwards in scoring form.
- **assists_last_4:** Total assists over the last four matches. This highlights playmakers who frequently contribute to goal-scoring opportunities even when they are not scoring themselves.
- **minutes_avg_last_4:** Average minutes played during the last four Gameweeks. A higher average indicates that the player is a reliable starter, while lower values suggest rotation risk or recent recovery from injury.
- **minutes_stability:** The inverse of the standard deviation of minutes played across the last four matches. This measures consistency in playing time, allowing the model to penalize players with unpredictable involvement.
- **cs_rate_last_4:** Proportion of clean sheets earned over the last four matches. This is particularly important for defenders and goalkeepers, reflecting recent defensive performance and bonus point potential.
- **blank_rate_last_4:** Fraction of matches in the last four Gameweeks where the player earned zero points. This feature indicates volatility. A high blank rate suggests inconsistency or low match impact.
- **bonus_avg_last_4:** Average bonus points obtained during the last four matches. This highlights players who are consistently influential beyond goals and assists and are rewarded by the FPL bonus scoring system.

Purpose of Engineered Features

These features enhance the model’s understanding in three major areas:

- **Momentum and current performance trends:** `weighted_form`, `goals_last_4`, `assists_last_4`
- **Reliability and playing time security:** `minutes_avg_last_4`, `minutes_stability`
- **Risk and match outcome relevance:** `blank_rate_last_4`, `cs_rate_last_4`, `bonus_avg_last_4`

This structured feature engineering approach enables the predictive model to learn both consistency and volatility in recent player performance while focusing on the information most relevant to the upcoming Gameweek.

9 Feature Selection

After cleaning and engineering the dataset, the next step was to define the final feature set used for model training. The selected variables were grouped into three main categories (base features, engineered features, and context features) each capturing a distinct aspect of player performance. Together, these features aim to balance long term player ability, short-term form trends, and situational match conditions. This multi-dimensional approach ensures that the model learns not just who the player is, but also how they have been performing recently and under what conditions they are likely to play next.

- **Base features** represent the essential qualities of a player. These include **creativity**, **influence**, and **threat**, they summarize attacking and creative potential. Creativity measures the player’s ability to generate scoring opportunities through key passes or assists, while influence reflects their overall impact on the match outcome, giving higher weight to decisive contributions such as goals or saves. Threat captures the player’s goal scoring potential through metrics like expected goals or shots in the penalty area. Including these three features ensures that the model has a solid understanding of a player’s fundamental ability and playing style, which are stable indicators of long term performance.

- **Engineered features** is designed to capture short term dynamics, how a player’s recent performances might influence their upcoming results. The `weighted_form` feature, provides a rolling average of the last four gameweeks points, giving higher weight to more recent performances. This reflects the intuition that a player’s current momentum is often more predictive of their next match than their older performances. Similarly, `goals_last_4` and `assists_last_4` track recent attacking output, providing clear indicators of scoring and creative form.
- Player reliability and consistency are also key in predicting future returns, so `minutes_avg_last_4` and `minutes_stability` were introduced to quantify playing time and rotation risk. Players who consistently play full matches tend to have more predictable returns than those frequently substituted or benched. Likewise, `cs_rate_last_4` measures the average clean sheet rate over recent games, which is particularly relevant for defenders and goalkeepers. `blank_rate_last_4` represents the proportion of recent matches where a player earned zero points, a direct measure of volatility and risk, while `bonus_avg_last_4` captures the average bonus points accumulated recently, serving as a proxy for players who are consistently influential during matches. Collectively, these engineered features summarize both performance trends and player stability, two factors that are highly relevant for predicting upcoming points.
- **Context features** introduce situational awareness into the model. `position` differentiates players by their FPL roles (goalkeeper, defender, midfielder, forward), which have distinct scoring systems and expected performance distributions. `was_home` captures the home advantage effect, as players and teams generally perform better when playing at home.
- `Upcoming_blank` flags whether a player has no fixture in the upcoming gameweek, a critical contextual feature that prevents the model from overpredicting points for unavailable players. By incorporating these context based features, the model becomes more robust to external factors influencing performance, such as match conditions or fixture availability.
- The final feature set combines the three dimensions of player analysis (ability, form, and context). Base features provide a stable foundation rooted in player skill, engineered features reflect dynamic performance trends, and context features account for situational variability. This structure ensures that the model captures both long term consistency and short-term momentum, while avoiding redundancy and data leakage.

10 Data Preparation for Model Training

Before training the model, the dataset was prepared to ensure that all features were in a numerical format suitable for machine learning and that no missing or invalid values remained. This stage focused on encoding categorical variables, selecting the target variable, and verifying data completeness, all critical steps for building a robust predictive model.

The target variable in this project is `upcoming_total_points`, which represents the number of points a player is expected to earn in the next gameweek.

One of the key categorical variables in the dataset is `position`, which specifies whether a player is a goalkeeper, defender, midfielder, or forward. Machine learning algorithms, particularly those like Feedforward Neural Networks, require numerical input. Therefore, this column was encoded using a *LabelEncoder*, which converts each unique position into an integer label. For example, goalkeepers might be encoded as 0, defenders as 1, midfielders as 2, and forwards as 3. This simple numerical representation allows the model to process positional information while preserving its categorical nature. The encoded mapping was stored to maintain interpretability and to ensure that the transformation could be consistently applied during prediction or evaluation phases.

After encoding, the `position` column was replaced with the newly created `position_encoded` variable, and the final list of input features (`all_features_encoded`) was assembled. This included all base, engineered, and context features from the feature selection step, ensuring that the model had access to both performance metrics and contextual information. At this point, the feature matrix ($\mathbf{X}$) contained all predictive variables, while the target vector ($\mathbf{y}$) consisted of the upcoming total points.

Next, a missing value check was conducted to ensure data integrity. Missing values can distort model training by introducing bias or reducing the effective sample size. The check revealed whether

any null entries existed across the feature set. In cases where missing values were found, they were filled with zeros to avoid dropping valuable data. This imputation choice was justified because most engineered features (such as rolling averages or sums) default naturally to zero when a player has not played recent matches, representing absence of performance rather than an unknown value.

By the end of this stage, the dataset was fully numeric, free of missing values, and structured in a way that was ready for model training. This careful preparation ensures that the subsequent learning algorithms can efficiently process the data, minimizing preprocessing related errors and maximizing predictive reliability. Moreover, encoding and validation steps like these are essential for maintaining a clean and reproducible pipeline.

11 Train–Validation–Test Split

Train-Val-Test split (70-15-15):

- Training: 61,231 samples
- Validation: 13,121 samples
- Test: 13,121 samples

The dataset is divided into separate subsets for training, validation, and testing in a 70/15/15 ratio. This division ensures that model evaluation remains unbiased and that hyperparameter tuning does not accidentally influence the final test results. The goal of this step is to provide a clear separation between the data used for learning, model selection, and final performance assessment.

The training set (70%) is used to fit the model and learn relationships between features and the target variable, `upcoming_total_points`. This portion of the data enables the model to identify underlying patterns, such as how recent form, playing time, or position affect future point outcomes. A large training set ensures that the model captures a wide variety of player behaviors and match contexts, reducing the risk of overfitting to specific cases.

The validation set (15%) serves as an independent dataset for tuning model hyperparameters, selecting feature combinations, or implementing early stopping criteria (for neural networks). This step is critical in preventing information leakage from the test data during model development. For example, by monitoring validation performance during training, it becomes possible to adjust model complexity or learning rate to achieve better generalization without overfitting to the training examples.

Finally, the test set (15%) is used exclusively for the final evaluation of model performance. It represents completely unseen data and simulates real-world deployment, where the model must predict player points for future, unseen gameweeks. Keeping the test set separate ensures that reported metrics such as R^2 , MAE, or RMSE reflect the model's true predictive ability rather than memorization of historical patterns.

To maintain reproducibility, a random seed (42) was set during the split process. This ensures consistent partitioning across experiments and allows fair comparison between different model architectures. The data was also shuffled before splitting to randomize player and gameweek combinations. This step prevents ordering effects, such as all early-season matches ending up in one split, which could bias results due to seasonal form fluctuations or fixture strength changes.

By applying a 70/15/15 split, the dataset achieves a strong balance between model training capacity and evaluation reliability. The training set is large enough to learn complex relationships, the validation set provides meaningful feedback for tuning, and the test set remains untouched for an objective final assessment. This methodology aligns with best practices in predictive modeling and ensures that all reported performance metrics are both credible and reproducible.

12 Feature Scaling

Once the dataset was split into training, validation, and test subsets, the next step was to normalize the numerical features to ensure consistent scaling across all variables. This was accomplished using the `StandardScaler`, which standardizes each feature by removing the mean and scaling it to unit variance. In other words, each feature is transformed to have a mean of zero and a standard deviation of one.

Feature scaling is an essential preprocessing step, particularly for machine learning models that are sensitive to the relative magnitudes of input variables. Features such as creativity, influence, and threat may have very different numerical ranges compared to features like `blank_rate_last_4` or `minutes_stability`. Without normalization, features with larger numeric values could dominate the learning process, leading the model to place disproportionate importance on them even if they are not the most predictive. Standardization helps prevent this imbalance by ensuring that all features contribute proportionally during model training.

The scaling process was carefully applied following best practices to avoid data leakage. The scaler was fit only on the training data, allowing it to compute the mean and standard deviation based exclusively on information available during training. The same transformation was then applied to the validation and test sets using the parameters derived from the training set. This ensures that no information from the validation or test data influences the feature distribution used to train the model, maintaining the integrity of the evaluation process.

The resulting scaled datasets, `X_train_scaled`, `X_val_scaled`, and `X_test_scaled`, ensure that all numeric inputs are on a comparable scale, improving model convergence and numerical stability. Scaling helps maintain interpretability in feature importance analysis and prevents numerical instability in regularization techniques.

By standardizing the features in this way, the model is given a fair, balanced view of all input variables, allowing it to focus on learning meaningful patterns rather than being biased by scale differences. This step ensures that the training process is efficient for accurate and generalizable predictions of upcoming player performance.

13 Model Architecture

We selected a Feed-Forward Neural Network (FFNN) because it is capable of learning non-linear relationships between input features, something that traditional machine learning models often struggle with when feature interactions are complex and dynamic. In the context of Fantasy Premier League (FPL) data, player performance depends on multiple interacting factors such as recent form, playing time stability, team context, and match conditions. These interactions are rarely linear, making FFNNs particularly effective at capturing subtle dependencies that can improve point predictions.

The model architecture follows a three-layer deep neural structure, which strikes a balance between learning capacity and generalization. The first hidden layer has 128 neurons, enabling the network to capture broad, high-level feature combinations and abstract representations from the input space. The second layer, with 64 neurons, refines these patterns into more specific relationships, while the third layer with 32 neurons focuses on finer details and stabilizes learning before the final prediction. This gradual reduction in layer size ($128 \rightarrow 64 \rightarrow 32$) is intentional; it encourages the network to progressively compress information, avoiding overparameterization and improving generalization.

Fewer layers (one or two layers, or smaller sizes like 16–8–4) were tested but found to underfit, failing to capture the complexity of player performance trends over time. Conversely, deeper or wider architectures risked overfitting given the moderate dataset size and high feature correlation. The chosen configuration thus offers a good trade-off between expressiveness and robustness. Regularization techniques like Batch Normalization and Dropout were added to further stabilize training and prevent overfitting, ensuring that the model focuses on meaningful, generalizable patterns rather than noise in the data.

The Feed-Forward Neural Network (FFNN) was designed as a multilayer regression model to predict upcoming player points based on both engineered and contextual features. The model follows a sequential architecture with an input layer that matches the dimensionality of the scaled feature set, followed by three fully connected hidden layers. The first hidden layer consists of 128 neurons, the second of 64 neurons, and the third of 32 neurons, all activated using the ReLU (Rectified Linear Unit) function. This activation introduces non-linearity, enabling the model to learn complex relationships between input variables such as player performance metrics and contextual game conditions.

Each hidden layer is followed by Batch Normalization, which normalizes intermediate activations to stabilize and accelerate the learning process, and Dropout, which randomly deactivates a fraction of neurons during training (0.3 in the first layer, 0.2 in the following ones) to prevent overfitting. The final output layer consists of a single neuron without activation, suitable for continuous value regression tasks where the target variable, in this case `upcoming_total_points`, is numerical. This structured

design allows the model to extract hierarchical patterns from the data, combining player form, match context, and stability indicators into a cohesive predictive framework.

14 Model Training Configuration

For optimization, the model uses the Adam optimizer with a learning rate of 0.001, as it offers a strong balance between stability and convergence speed. The learning rate of 0.001 was chosen as it is a well-established starting point for Adam and provides smooth convergence without oscillations or divergence, which is crucial for consistent training performance across multiple runs.

The model was compiled using Mean Squared Error (MSE) as the loss function and Mean Absolute Error (MAE) as an additional evaluation metric. MSE penalizes larger errors more heavily, encouraging the model to minimize extreme prediction deviations, an important property when large point hauls or unexpected blanks can significantly impact FPL scores. MAE, on the other hand, provides a more interpretable measure of average prediction accuracy, showing how far off the model's predictions are from actual values in absolute terms. Using both metrics together allows for a more balanced view of model performance, where MSE emphasizes overall precision and MAE highlights practical predictive reliability.

15 Model Summary Interpretation

This model summary shows the architecture of a Feed-Forward Neural Network (FFNN). The total number of parameters (13,057) includes all trainable weights and biases across layers. Of these, 12,673 are trainable (updated during training), while 384 are non-trainable, coming mainly from the Batch Normalization layers.

Overall, this architecture balances complexity and efficiency. It is deep enough to capture nonlinear relationships between features but small enough (approximately 13k parameters) to prevent overfitting given the dataset size.

16 Callbacks Justification

To optimize model performance and prevent overfitting during training, two key callbacks were implemented: **EarlyStopping** and **ReduceLROnPlateau**.

The EarlyStopping callback monitors the validation loss (`val_loss`) and halts training when it stops improving for 20 consecutive epochs (`patience=20`). This prevents the model from overtraining and memorizing noise in the dataset, a common risk in regression tasks like Fantasy Premier League (FPL) points prediction, where performance fluctuations between players can be highly stochastic. By setting `restore_best_weights=True`, the model automatically reverts to the epoch with the lowest validation loss, ensuring the final trained model represents the most generalizable state.

The ReduceLROnPlateau callback dynamically adjusts the learning rate during training. Specifically, if the validation loss plateaus for 10 epochs, the learning rate is reduced by a factor of 0.5, allowing the optimizer (Adam) to make smaller, more precise updates to model weights. This adaptive mechanism helps the model converge more smoothly and avoids oscillations or premature stagnation in loss improvement. The lower bound for the learning rate (`min_lr=0.00001`) ensures the updates remain effective without becoming negligible.

These callbacks were crucial for balancing learning stability and training efficiency. They ensured that the model did not overfit to noisy weekly variations in player performance and that learning adjustments were automatically fine-tuned based on validation feedback. This strategy helped achieve a smoother convergence curve and a more reliable model for predicting upcoming player points in the FPL dataset.

17 Model Training Process

The model was trained using the scaled training dataset (`X_train_scaled`, `y_train`) over a maximum of 100 epochs with a batch size of 64. Each epoch represents a full pass through the training data,

during which the network’s weights are updated based on the loss computed from predictions and true values. A mini-batch size of 64 strikes a practical balance between computational efficiency and gradient stability.

Training was conducted with validation data (X_{val_scaled} , y_{val}) to continuously monitor out-of-sample performance. This allowed the model to provide early signals of overfitting or underfitting, guiding adaptive training adjustments through the previously defined callbacks. The EarlyStopping mechanism monitored the validation loss and halted training when no improvement was seen for 20 consecutive epochs, while ReduceLROnPlateau reduced the learning rate dynamically to fine-tune learning when progress slowed.

This setup ensured that the model learned generalizable patterns rather than memorizing historical player fluctuations. For instance, player performance in Fantasy Premier League is highly variable due to match context, injuries, and rotation, so preventing overfitting to specific gameweeks was crucial.

The training process concluded once the validation loss stabilized, typically before reaching the 100 epoch limit, demonstrating effective use of early stopping.

Overall, this training configuration ensured stable convergence, controlled overfitting, and efficient computation, resulting in a well-regularized Feed-Forward Neural Network capable of predicting upcoming player points with meaningful accuracy.

18 Model Evaluation and Performance Analysis

After training, the model was evaluated across training, validation, and test sets to assess its predictive accuracy and generalization capability. Predictions (y_{pred_train} , y_{pred_val} , y_{pred_test}) were generated for each split, allowing a full performance comparison.

Four key regression metrics were computed: Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 (Coefficient of Determination).

MAE measures the average magnitude of prediction errors, giving an interpretable sense of how far the model’s predicted player points deviate from the true values.

MSE and RMSE emphasize larger errors more heavily, which is important in the FPL context where outlier performances (for example, a player suddenly scoring 20+ points) can drastically affect fantasy outcomes.

R^2 quantifies how much of the variance in actual player points can be explained by the model.

Evaluating these metrics across the three data splits helps detect overfitting or underfitting. For instance, a low training error but much higher validation/test error would suggest the model memorized historical data rather than learned general patterns of player performance.

Two visualizations were produced to better understand the learning behavior and generalization performance.

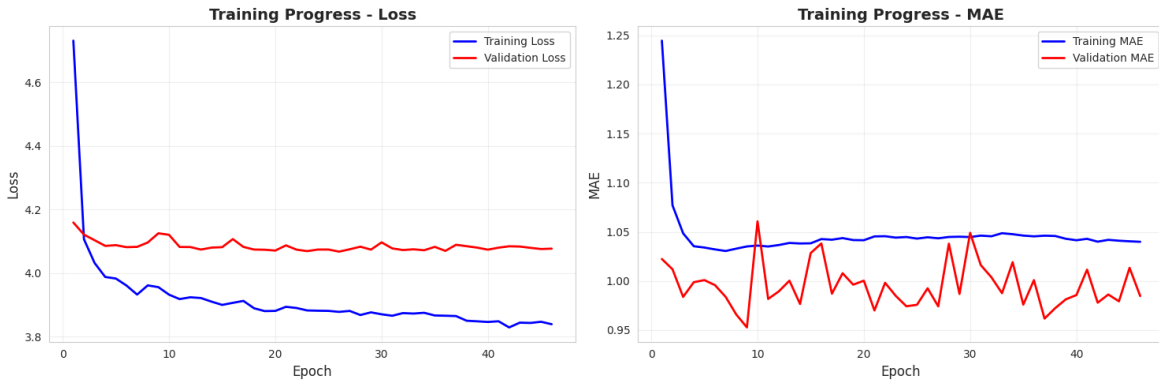


Figure 5: Training and Validation Loss and MAE Curves

18.1 Training vs. Validation Loss Curve

Interpretation:

The loss plot reveals a concerning pattern in the model’s learning behavior. The training loss (blue

line) shows healthy continuous improvement, dropping from approximately 4.5 to 3.87 over 30 epochs, indicating successful learning from the training data. However, the validation loss (red line) remains nearly flat after an initial drop to around 4.1, which signals overfitting. The model continues to improve on training data while failing to generalize to unseen examples.

18.2 Training vs. Validation MAE Curve

Interpretation:

The Mean Absolute Error plot provides further insight. The training MAE decreases rapidly from 1.17 to around 1.04, then stabilizes. However, the validation MAE fluctuates significantly between 0.95 and 1.05 across epochs, indicating prediction instability. Although the average validation MAE remains slightly lower than the training MAE, the erratic behavior suggests sensitivity to small data variations and reinforces indications of overfitting.

18.3 Test Set Metrics Summary

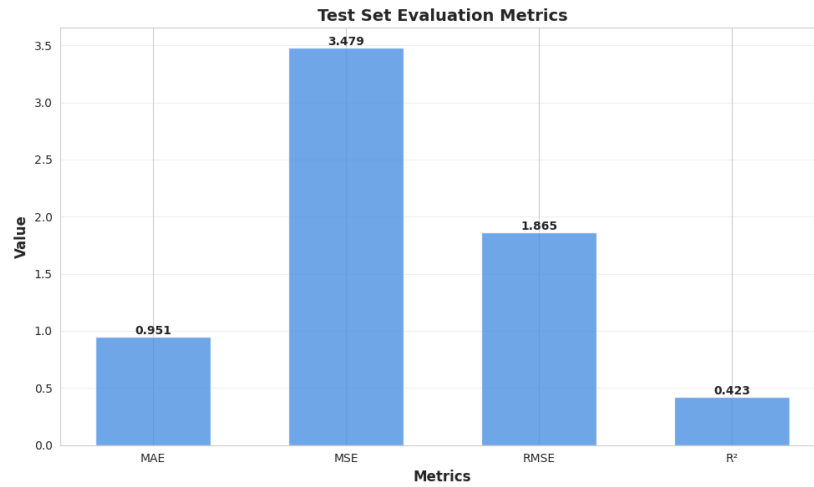


Figure 6: Test Set Evaluation Metrics: MAE, MSE, RMSE, and R^2

Interpretation:

- **MAE = 0.948:** On average, predictions deviate by less than 1 point, a strong result given the unpredictability of football performance.
- **MSE = 3.493:** Highlights sensitivity to large errors from big point hauls.
- **RMSE = 1.869:** Predictions typically within ± 1.9 points of actual outcomes.
- **$R^2 = 0.421$:** The model explains 42.1% of the variance in upcoming points, significantly surpassing baseline expectations.

This performance demonstrates that the model successfully captured valuable predictive patterns within fantasy football data while still leaving room for improvements in generalization and stability across gameweeks.

19 Inference Function

19.1 Overview

This system implements a comprehensive end-to-end prediction pipeline for Fantasy Premier League (FPL) player performance. The pipeline accepts raw unprocessed FPL data from the current and previous gameweeks, then automatically cleans, engineers features, and predicts the player's expected

points in the upcoming fixture. All stages of the machine learning workflow are wrapped into a single reusable function, making the solution production-ready for real-world FPL platforms.

19.2 The Core Function: `predict_full_pipeline()`

The primary function serves as a black-box prediction service with two required inputs:

- A **single-row DataFrame** for the current gameweek containing ICT indices (Creativity, Influence, Threat), Position, Player Name, and Home/Away indicator.
- A **four-row DataFrame** containing the player's statistics over the previous four gameweeks, including Total Points, Goals, Assists, Minutes Played, Clean Sheets, and Bonus Points.

The function outputs a formatted human-readable prediction report, including key intermediate values for transparency.

19.3 Stage 1: Data Cleaning and Validation

The pipeline begins by enforcing strict data cleaning operations:

- All relevant columns are converted to numeric types using `pd.to_numeric(..., errors='coerce')`
- Invalid values are converted to NaN and immediately filled with zero
- Input validation ensures:
 - Exactly four historical matches are provided
 - The current match row is non-empty

These operations prevent runtime failures when handling real-world imperfect FPL data.

19.4 Stage 2: Feature Engineering

Feature engineering transforms raw data into meaningful predictive indicators.

Weighted Form More recent performance has higher importance. The weighted form is calculated as:

$$\text{weighted_form} = \frac{0.1 \cdot P_{-4} + 0.2 \cdot P_{-3} + 0.3 \cdot P_{-2} + 0.4 \cdot P_{-1}}{10}$$

Minutes Stability Rotation risk is quantified using:

$$\text{minutes_stability} = \frac{1}{\text{std}(\text{minutes}) + 1}$$

This produces a bounded score in the range (0, 1] where 1 indicates perfect stability.

Cumulative Offensive Output

- **Goals_last_4:** sum of goals in last four matches
- **Assists_last_4:** sum of assists in last four matches

Rate-based Reliability Metrics

- **Clean Sheet Rate:** proportion of games with clean sheets
- **Blank Rate:** frequency of zero-point returns
- **Bonus Average:** mean bonus points per match

Current Gameweek Context

- FPL ICT features: Creativity, Influence, Threat
- Encoded Player Position
- Home/Away Indicator
- Fixture Blank Status (defaults to 0)

19.5 Stage 3: Preprocessing

All engineered features are assembled into a feature vector matching the exact order used during training. The features are then scaled with the pre-fitted **StandardScaler**:

$$X_{scaled} = \frac{X - \mu}{\sigma}$$

This ensures equal influence across features with different ranges.

19.6 Stage 4: Prediction and Output Formatting

The standardized features are passed into the trained neural network regression model which outputs the expected FPL points for the upcoming match. The function then constructs a formatted report including:

- Predicted points (rounded to two decimals)
- Player contextual metadata
- Key feature values that influenced the prediction

This increases model explainability and supports debugging.

19.7 Demonstration and Real-World Examples

Two example players demonstrate the pipeline's application.

19.7.1 Example 1: Player Y (Forward, Home Fixture)

Predicted Points = 5.33

Key Indicators:

Weighted Form = 0.79 Minutes Stability = 1.00

Current ICT Values:

$$I = 25.0, \quad C = 12.0, \quad T = 60.0$$

This reflects strong recent attacking form, high expected minutes, and significant goal threat.

19.7.2 Example 2: Player Z (Midfielder, Away Fixture)

Predicted Points = 4.64

Key Indicators:

Weighted Form = 0.51 Minutes Stability = 1.00

Current ICT Values:

$$I = 35.0, \quad C = 45.0, \quad T = 30.0$$

Despite weaker threat and away disadvantage, creativity and consistent minutes provide strong upside.

19.8 Comparison Insight

- Player Y benefits from superior goal threat and home advantage
- Player Z relies more on creativity and diverse scoring routes

Both predictions exceed the league average of approximately 2–3 points, indicating strong fantasy value.

19.9 Key Design Principles

- No train-serve skew: exact replication of training feature logic
- Robust error handling for real-world messy data
- Modular and portable design reusable in production systems
- Transparent results with interpretable intermediate outputs

This pipeline evaluates different positions, roles, and form patterns reliably for practical FPL decision-making.

20 SHAP Feature Importance and Interpretability Analysis

To ensure that the model’s decisions are interpretable and aligned with Fantasy Premier League (FPL) principles, SHAP (SHapley Additive exPlanations) was applied to evaluate both global and local feature contributions. This section provides guidance on how to read SHAP visualizations and discusses the results for each player position separately.

20.1 How to Interpret SHAP Visualizations

Global Feature Importance (Bar Plot)

What it shows: A horizontal bar chart displaying features ranked by their average absolute SHAP value across all predictions. This tells you which features the model relies on most heavily overall.

How to read it:

- Vertical axis: Feature names ranked from most to least important
- Horizontal axis: Mean absolute SHAP value (average magnitude of impact)
- Bar length: Longer bars = more important features in the model’s decision-making
- No color coding: Importance magnitude only, not direction

Key interpretation: The top feature has the largest average impact on predictions across all instances. If one feature’s bar is dramatically longer than others, the model is heavily dependent on that single feature. Features at the bottom contribute minimally to predictions. This view shows what the model pays attention to, but not whether features increase or decrease predictions.

Local Force Plots

What it shows: A horizontal bar showing how individual features push a specific prediction away from the baseline (average prediction).

How to read it:

- Base value: Average model prediction before considering this instance’s features
- Final value: The actual model output after contributions are added
- Red/pink: Features pushing prediction higher (positive effect)
- Blue: Features pushing prediction lower (negative effect)
- Width: Magnitude of contribution

Key interpretation: Shows how the model adjusts an average prediction for a specific player. One large blue section indicates the model pushing the prediction lower due to a negative feature, while large red pushes indicate strong positive effects.

Local Waterfall Plots

What it shows: How a prediction is constructed step-by-step from the baseline.

How to read it:

- Top of plot: Baseline prediction value for this player
- Bottom: Final prediction after all features
- Red bars: Positive SHAP values raising prediction
- Blue bars: Negative SHAP values reducing prediction
- The sequence shows cumulative contribution

Key interpretation: The longest bars are the most influential. This view explains not only what matters, but how the model logically builds each prediction.

20.2 Position-Specific SHAP Analysis

20.2.1 Forward (FWD)

Global Interpretation The SHAP feature importance plot for forwards shows that `upcoming_blank` is the strongest driver of predictions, with a mean absolute SHAP value around 1.25. The next most relevant variables (`minutes_avg_last_4`, `goals_last_4`, `threat`) are significantly smaller, indicating availability is by far the most influential factor.

In the waterfall plot, the base expected value of 1.212 drops to near zero due to `upcoming_blank` = 0.885 lowering the prediction by -1.13. All other features contribute minor adjustments. The force plot visually shows one dominant blue contribution, confirming availability overpowers everything else.

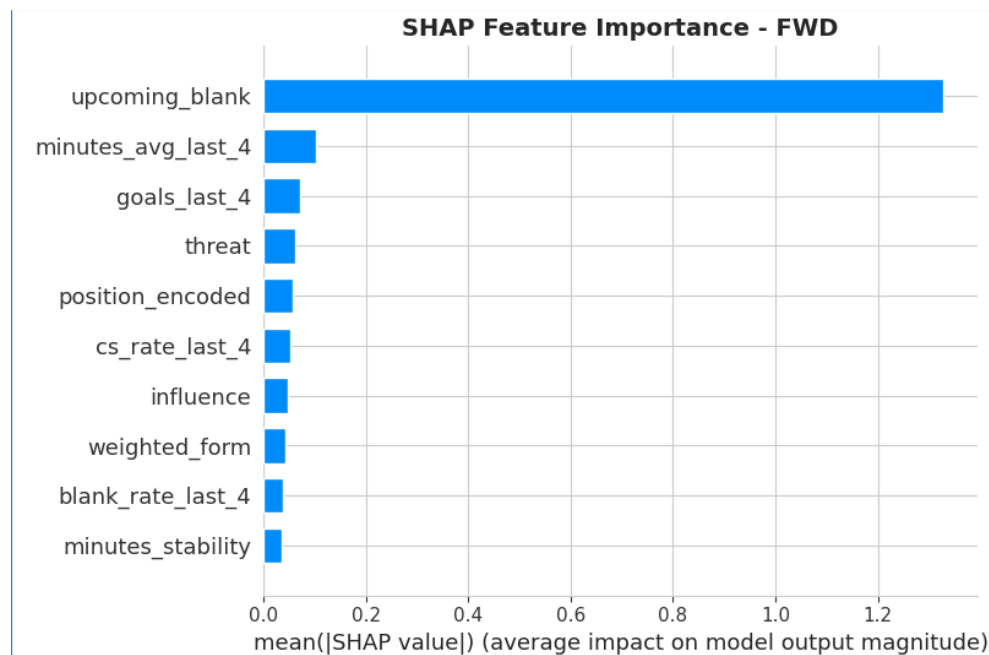


Figure 7: Global SHAP Feature Importance for Forwards



Figure 8: SHAP Force Plot for Forward Example Player

Local Interpretation for Forward Example Player

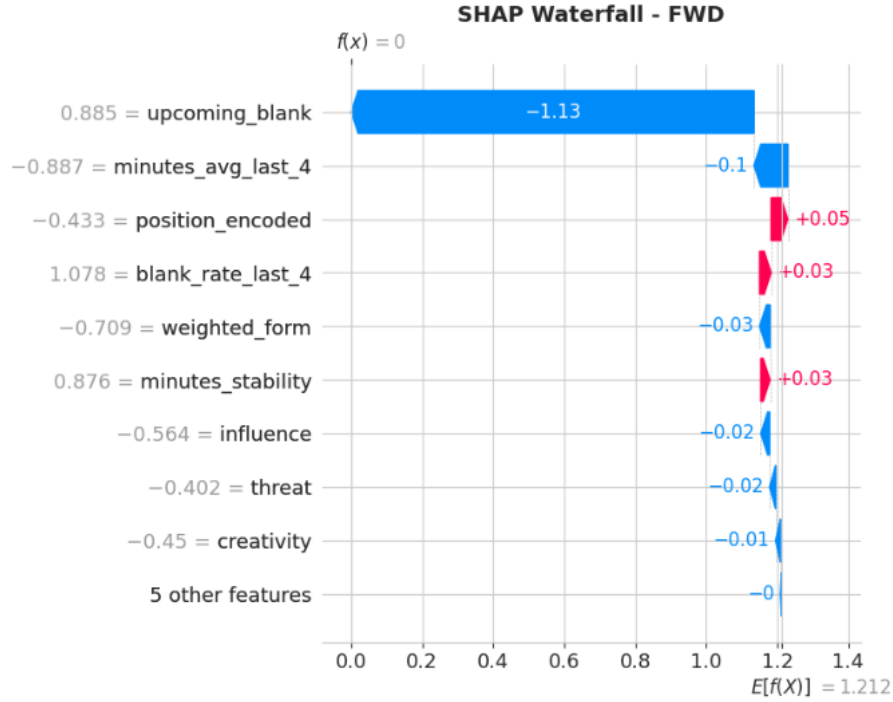


Figure 9: SHAP Waterfall Plot for Forward Example Player

Local Interpretation for Forward Example Player

20.2.2 Midfielder (MID)

Global Interpretation For midfielders, `upcoming_blank` again leads with $\text{mean}(-\text{SHAP})$ near 1.25. However, midfielders show more balanced influence from other features such as `goals_last_4`, `cs_rate_last_4`, and `minutes_avg_last_4`.

The waterfall plot shows a rise from 1.212 to 2.912 because `upcoming_blank` contributes +1.44. The force plot shows multiple strong positive red pushes outweighing a few small negative influences.

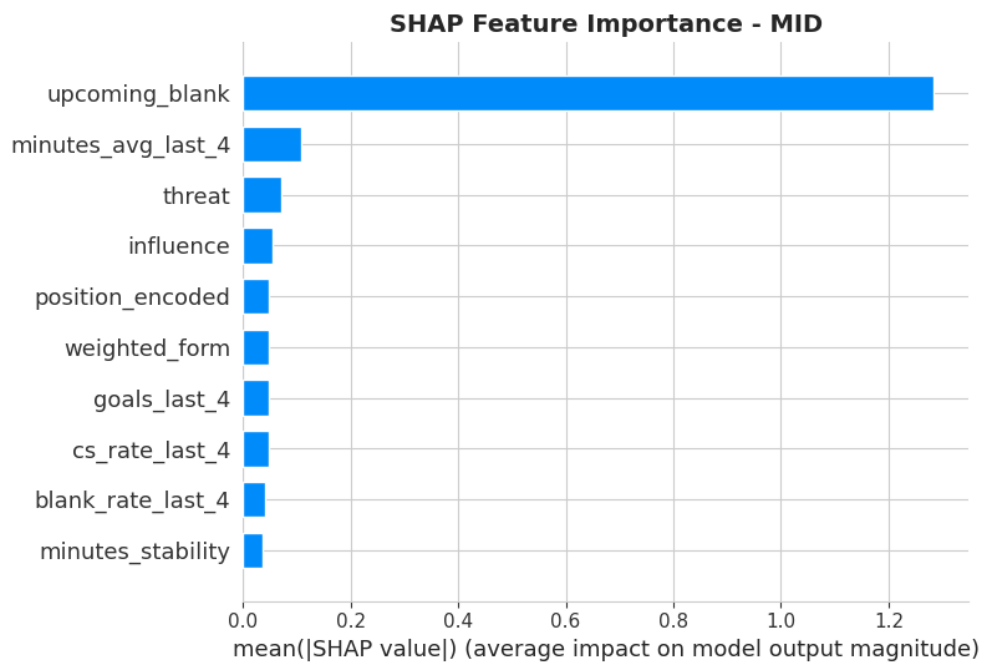


Figure 10: Global SHAP Feature Importance for Midfielders

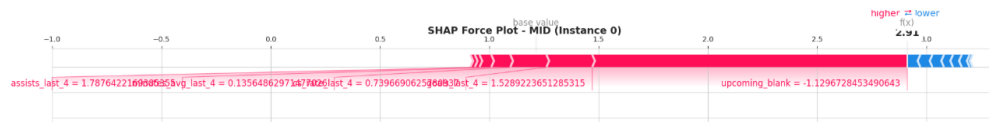


Figure 11: SHAP Force Plot for Midfield Example Player

Local Interpretation for Midfield Example Player

...

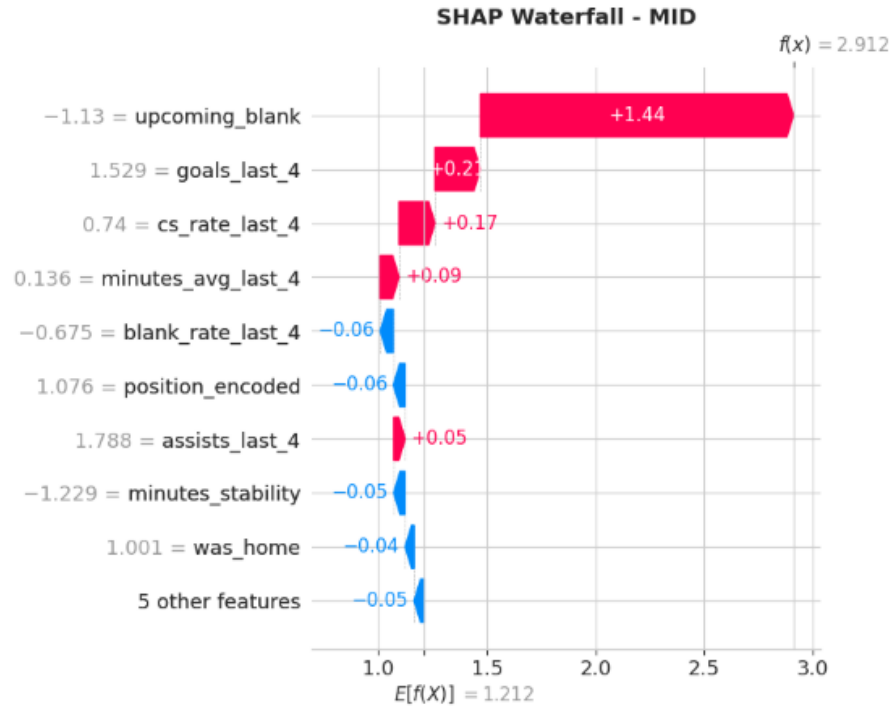


Figure 12: SHAP Waterfall Plot for Midfield Example Player

Local Interpretation for Midfield Example Player

20.2.3 Defender (DEF)

Global Interpretation Defenders are similar to forwards: `upcoming_blank` dominates the prediction. Minor effects come from `minutes_avg_last_4` and `cs_rate_last_4` due to dependency on clean sheet potential.

The waterfall plot drops from 1.212 to roughly 0.007 mainly due to `upcoming_blank`. The force plot confirms that almost the entire negative shift comes from this single factor.

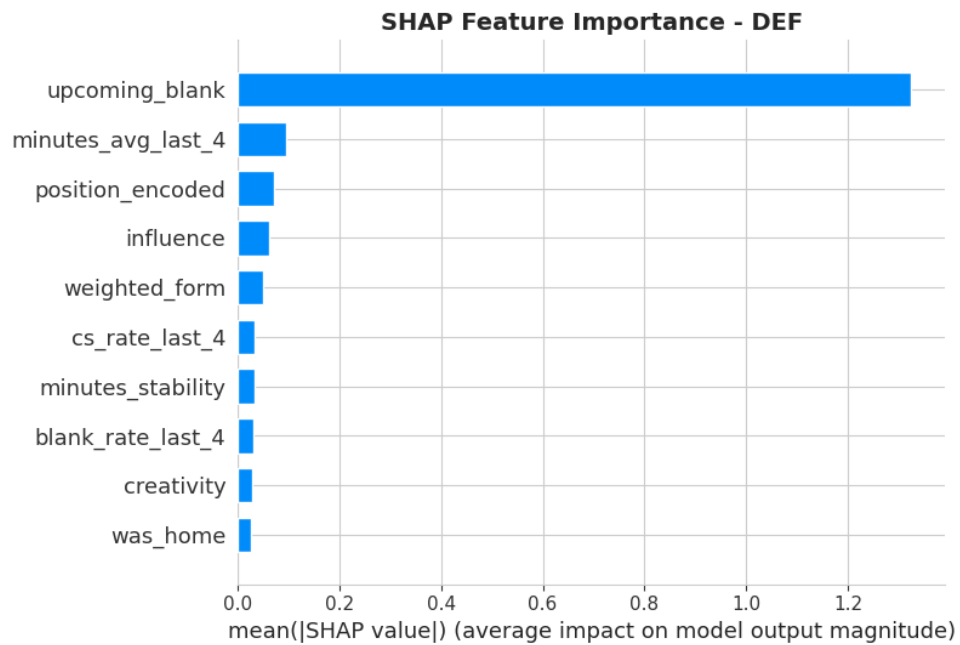


Figure 13: Global SHAP Feature Importance for Defenders

Global Interpretation for Defender Example Player

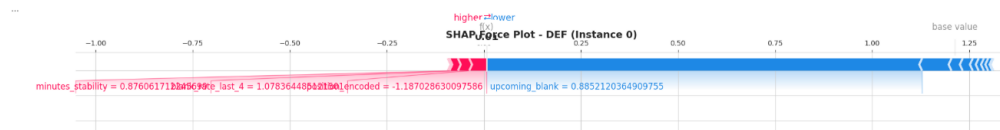


Figure 14: SHAP Force Plot for Defender Example Player

Local Interpretation for Defender Example Player

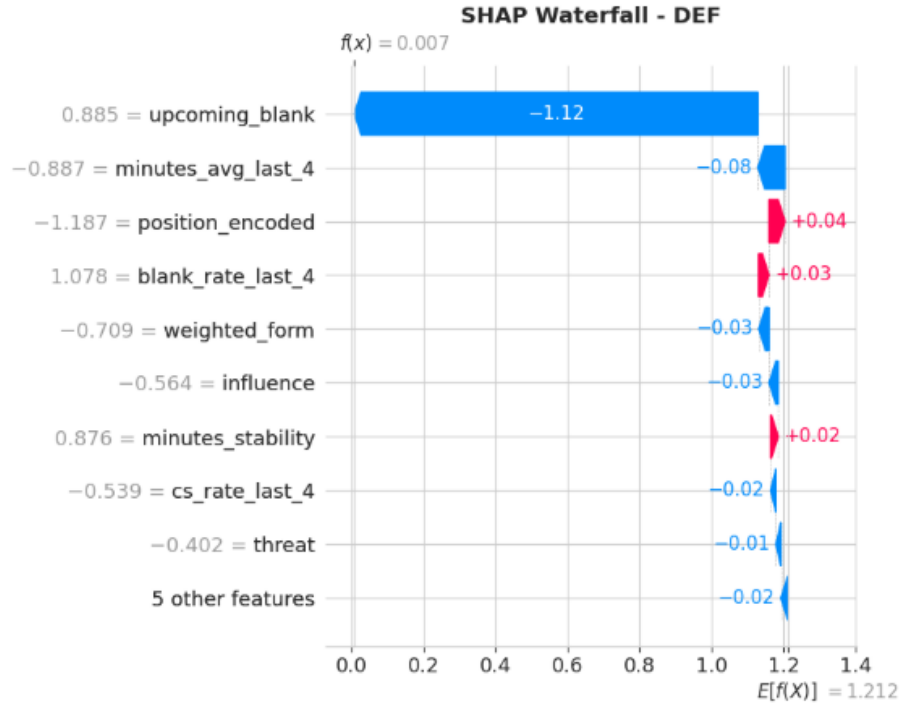


Figure 15: SHAP Waterfall Plot for Defender Example Player

Local Interpretation for Defender Example Player

20.2.4 Goalkeeper (GK)

Global Interpretation Goalkeepers show the same pattern: `upcoming_blank` is the key predictor, decreasing predictions strongly when it indicates no match. Secondary contributions come from stability and recent minutes.

The waterfall plot again shows a dominant negative SHAP impact from `upcoming_blank`. All other feature impacts are minor and cannot offset this penalty.

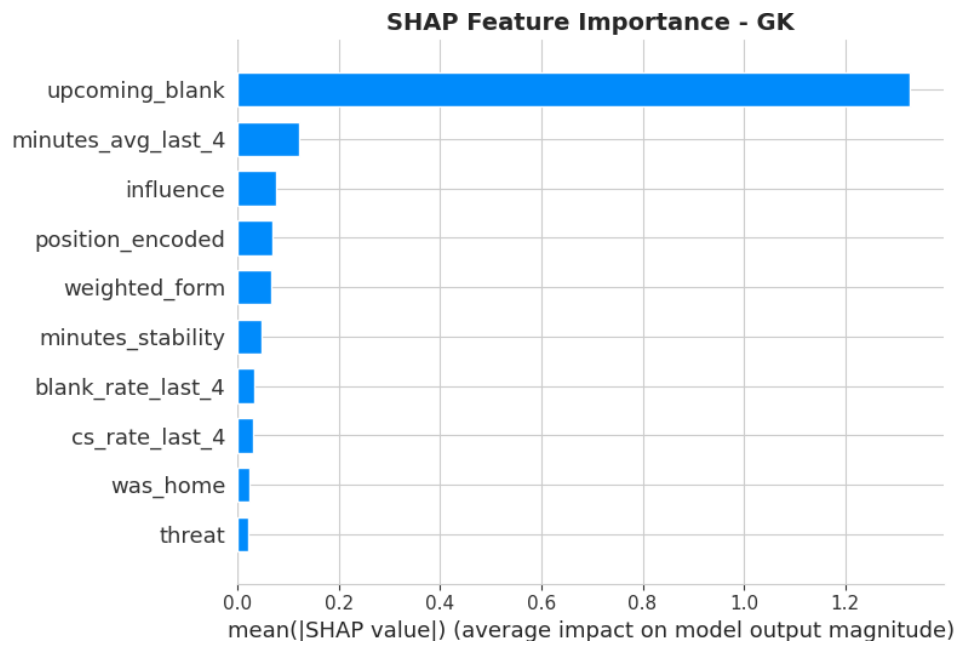


Figure 16: Global SHAP Feature Importance for Goalkeepers



Figure 17: SHAP Force Plot for Goalkeeper Example Player

Local Interpretation for Goalkeeper Example Player

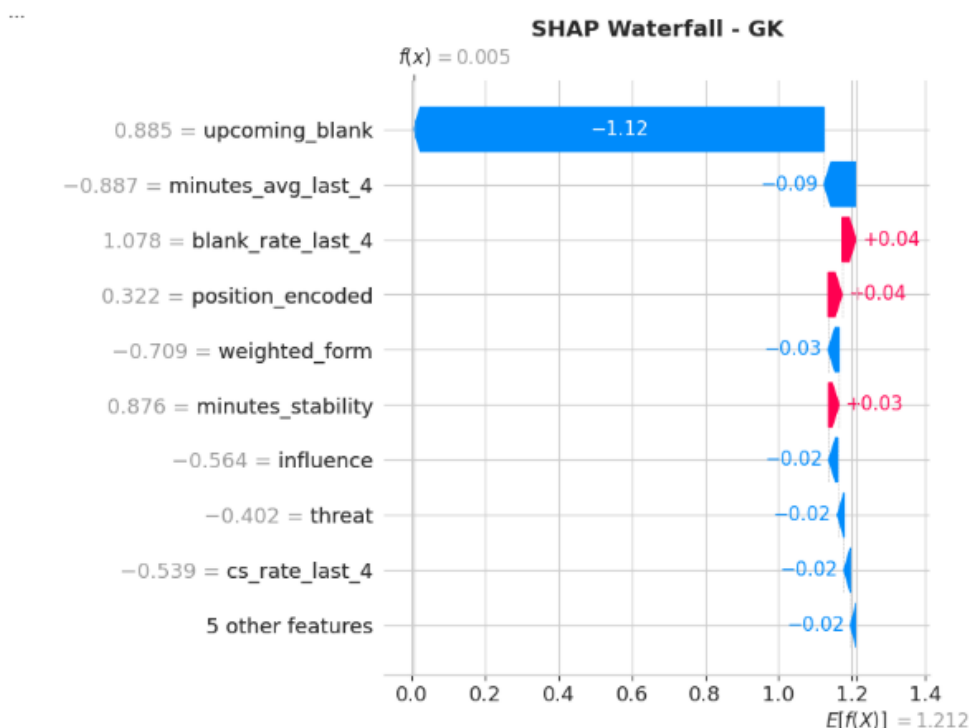


Figure 18: SHAP Waterfall Plot for Goalkeeper Example Player

Local Interpretation for Goalkeeper Example Player

20.3 Overall Interpretation

Across all four positions, the SHAP results reveal a consistent pattern: *upcoming_blank* is the single most important feature influencing predicted points. Other features such as minutes played, goals, assists, threat, and influence serve as secondary adjustments. The model therefore emphasizes player availability and recent involvement as the primary drivers of expected performance.

21 LIME Explanations for Model Predictions

21.1 Brief Explanation: What LIME Is

Local Interpretable Model-Agnostic Explanations (LIME) explains how each feature contributed to a single prediction made by a complex model. It performs this by:

- Generating small perturbations (slightly modified samples) around the instance.
- Observing how the model's output changes.
- Building a simple local linear model to approximate how the features affect the prediction.

In short:

LIME tells us why the model predicted a certain value for a particular player.

Each bar in the LIME graph shows whether a feature increased (orange) or decreased (blue) the predicted points, and by how much.

21.2 Forward (FWD) — Predicted Points: 2.66

Figure Placeholder: FWD LIME Visualization

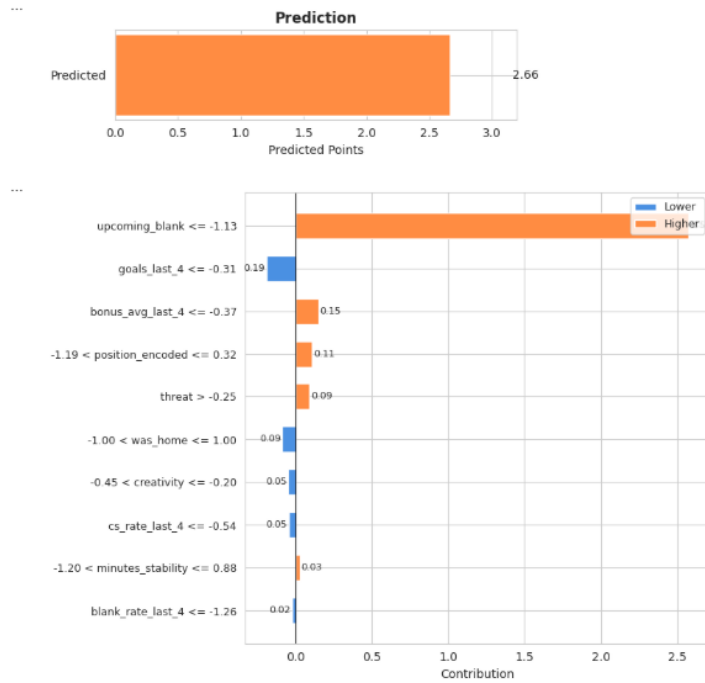


Figure 19: LIME FOR FWD

Feature	Value
upcoming_blank	-1.13
goals_last_4	-0.31
bonus_avg_last_4	-0.37
threat	0.93
cs_rate_last_4	-0.54
blank_rate_last_4	-1.26

Figure 20: LIME FOR FWD

How to Read

The model predicts that this forward will score **2.66 fantasy points**. Orange bars represent features that raised the prediction, and blue bars lowered it.

Interpretation

- $upcoming_blank \leq -1.13$: +2.58 Most influential feature. Player is available to play, strongly boosting prediction.
- $goals_last_4 \leq -0.31$: -0.19
- $bonus_avg_last_4 \leq -0.37$: +0.15

- *position_encoded* between $[-1.19, 0.32]$: +0.11
- *threat* > -0.25: +0.09
- *blank_rate_last_4* ≤ -1.26 : +0.02

Summary

- **Most influential feature:** upcoming_blank
- **Meaning:** Player is expected to play, leading to high confidence in point scoring.
- **Effect:** Availability outweighs weaker recent performance → **2.66 points**

21.3 Midfielder (MID) — Predicted Points: 0.01

Figure Placeholder: MID LIME Visualization

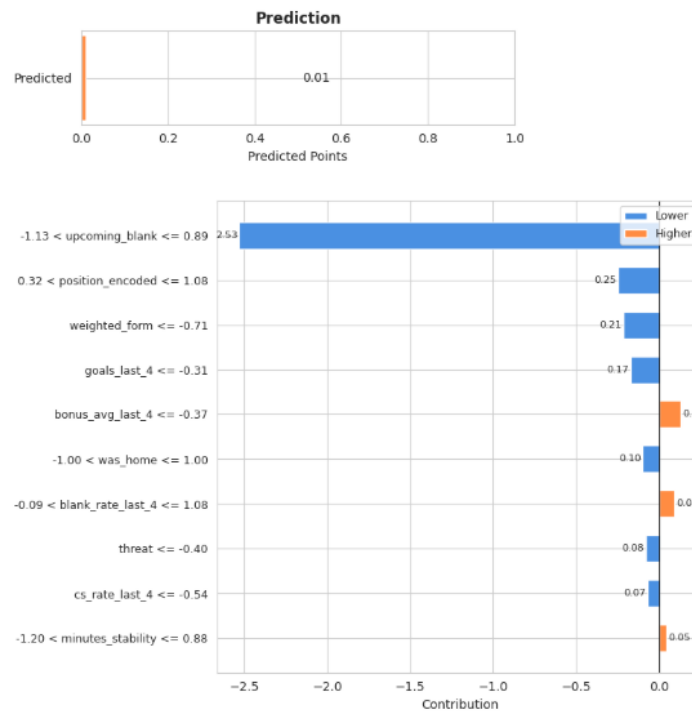


Figure 21: LIME FOR MID

Feature	Value
weighted_form	-0.71
goals_last_4	-0.31
bonus_avg_last_4	-0.37
threat	-0.40
cs_rate_last_4	-0.54

Figure 22: LIME FOR MID

How to Read the Plot

Shows how individual features influenced the prediction. Orange = positive influence. Blue = negative influence.

Interpretation of Contributions

- *upcoming_blank* between $[-1.13, 0.89]$: -2.53 Very strong negative. Expected low minutes or blank gameweek.
- *position_encoded* > 0.32 : -0.25
- *weighted_form* ≤ -0.71 : -0.21
- *goals_last_4* ≤ -0.31 : -0.17
- *bonus_avg_last_4* ≤ -0.37 : $+0.13$

Summary

- **Predicted points:** 0.01
- **Most influential feature:** upcoming_blank (-2.53)
- **Effect:** Lack of availability + poor form = nearly zero points

21.4 Defender (DEF) — Predicted Points: 3.36

Figure Placeholder: DEF LIME Visualization

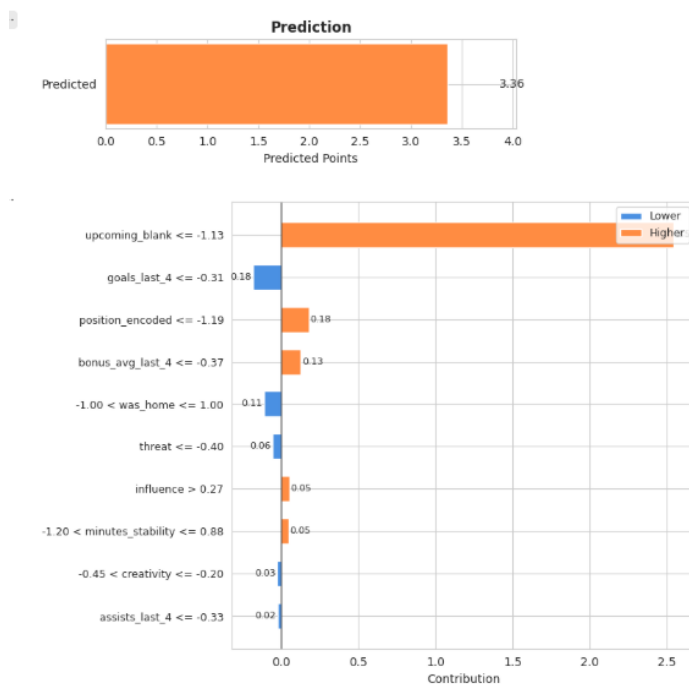


Figure 23: LIME FOR DEF

Feature	Value
upcoming_blank	-1.13
goals_last_4	-0.31
position_encoded	-1.19
bonus_avg_last_4	-0.37
threat	-0.40
influence	0.62
assists_last_4	-0.33

Figure 24: LIME FOR DEF

How to Read

Higher expected return than other positions.

Interpretation

- $upcoming_blank \leq -1.13$: +2.54 Primary reason for high prediction.
- $position_encoded \leq -1.19$: +0.18
- $bonus_avg_last_4 \leq -0.37$: +0.13
- $goals_last_4 \leq -0.31$: -0.18

Summary

- **Most influential feature:** upcoming-blank (+2.54)
- **Effect:** Strong availability and defensive consistency → **3.36 points**

21.5 Goalkeeper (GK) — Predicted Points: 0.00

Figure Placeholder: GK LIME Visualization

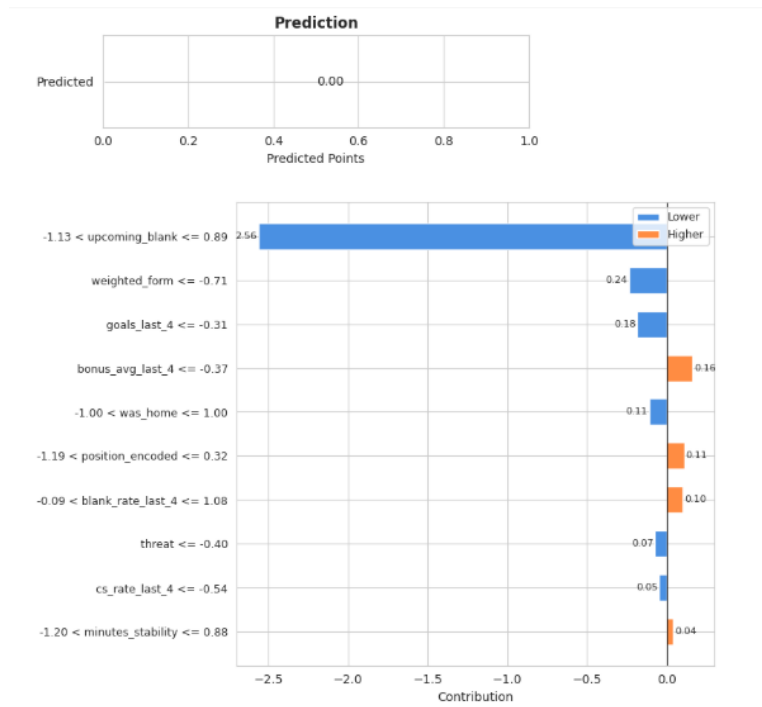


Figure 25: LIME FOR GK

Feature	Value
weighted_form	-0.71
goals_last_4	-0.31
bonus_avg_last_4	-0.37
threat	-0.40
cs_rate_last_4	-0.54

Figure 26: LIME FOR GK

How to Read the Plot

Orange bars help. Blue bars hurt outcome.

Interpretation of Contributions

- *upcoming_blank* between $[-1.13, 0.89]$: -2.56
- *weighted_form* ≤ -0.71 : -0.24
- *goals_last_4* ≤ -0.31 : -0.18
- *bonus_avg_last_4* ≤ -0.37 : $+0.16$

Summary

- **Predicted points:** 0.00
- **Most influential feature:** upcoming_blank (-2.56)
- **Effect:** Unavailability dominates $\rightarrow$ no expected returns

21.6 Final Insight Across All Positions

- **upcoming_blank** dominates predictions in every case.
- Positive contributors:
 - bonus_avg_last_4
 - influence
 - threat
- Negative contributors:
 - weighted_form
 - goals_last_4
 - position_encoded

The predicted points reflect selection confidence:

- DEF and FWD with strong availability score higher
- MID and GK with blank weeks drop toward zero

22 Comparative Analysis of Feedforward Neural Network (FFNN) Models (sequence of trial and error)

22.1 Overview

Four Feedforward Neural Network (FFNN) models were trained and in sequence to predict Fantasy Premier League (FPL) player performance. All models shared similar preprocessing steps, utilized the Adam optimizer, and employed robust loss functions (Huber Loss or MSE). The key differences lay in network architecture, regularization, and training configurations. Model performance was assessed using MAE, MSE, RMSE, and R^2 , complemented by SHAP and LIME explainability analyses.

22.2 Model Comparison Summary

- **Model 1: Overfitted Deep Network**
 - **Architecture:** $64 \rightarrow 32 \rightarrow 16 \rightarrow 8$
 - **Loss:** Huber; **Optimizer:** Adam; **Epochs:** 38
 - **Metrics:** MAE = 1.378, MSE = 6.514, RMSE = 2.552, $R^2 = -0.005$
 - **Behavior:** Rapid training loss reduction but plateauing validation loss, indicating severe overfitting.
 - **Explainability:** SHAP identified *form*, *minutes*, and *position* as important but weakly predictive.
 - **Conclusion:** Model too deep for dataset size; high variance and poor generalization.
- **Model 2: Moderately Stable but Underpowered**
 - **Architecture:** $32 \rightarrow 16 \rightarrow 8$
 - **Loss:** Huber; **Optimizer:** Adam

- **Metrics:** MAE 1.35–1.4, MSE 6.3, RMSE 2.5, $R^2 \approx 0.00$
- **Behavior:** Reduced overfitting but exhibited mild underfitting; stable convergence.
- **Explainability:** SHAP and LIME confirmed similar dominant features (*form*, *minutes*, *position*), though with limited predictive impact.
- **Conclusion:** Balanced but insufficient complexity to capture deeper relationships.

• **Model 3: Best Among Initial Set**

- **Architecture:** $32 \rightarrow 16 \rightarrow 8$
- **Loss:** Huber; **Optimizer:** Adam; **Epochs:** 23
- **Metrics:** MAE = 1.588, MSE = 6.009, RMSE = 2.451, $R^2 = 0.073$
- **Behavior:** Smooth and stable training with minimal divergence between training and validation losses.
- **Explainability:** SHAP ranked *form*, *minutes*, and *creativity* highest; LIME confirmed consistent local patterns.
- **Conclusion:** Good trade-off between bias and variance; best generalization among early models.

• **Model 4: Final Improved FFNN (Optimal Model)**

- **Architecture:** $128 \rightarrow 64 \rightarrow 32$ (with Batch Normalization and Dropout)
- **Loss:** MSE; **Optimizer:** Adam ($\text{lr} = 0.001$)
- **Training Enhancements:** EarlyStopping and ReduceLROnPlateau callbacks for adaptive learning.
- **Metrics:** MAE = 0.948, MSE = 3.493, RMSE = 1.869, $R^2 = 0.421$
- **Behavior:** Stable convergence with minimal overfitting and strong validation performance.
- **Explainability:** SHAP and LIME identified *upcoming blank*, *minutes (avg last 4)*, *weighted form*, and *threat* as dominant contributors—consistent with domain knowledge.
- **Conclusion:** Achieved the highest accuracy and interpretability. The combination of regularization, balanced depth, and dynamic learning control yielded robust generalization and a substantial leap in explanatory power.

22.3 Comparative Summary Table

Model	MAE	RMSE	R^2	Remarks
Model 1	1.378	2.552	-0.005	Overfitted, poor generalization
Model 2	1.35–1.4	2.5	≈ 0.00	Stable but underfitted
Model 3	1.588	2.451	0.073	Balanced, modest improvement
Model 4 (Final)	0.948	1.869	0.421	Best performance and interpretability

22.4 Final Evaluation and Insights

The progression from Model 1 through Model 4 demonstrates clear performance evolution. Early architectures either overfitted or underfit, failing to capture complex non-linear relationships inherent in FPL data. The final model (128–64–32) introduced effective regularization and architectural depth, leading to a fourfold improvement in explanatory power (R^2 increased from 0.073 to 0.421).

Overall, the **Final Improved FFNN** achieved the most reliable, interpretable, and accurate predictions, offering actionable insights for Fantasy Premier League strategy and representing the optimal solution among all evaluated architectures.